STUDENT RECORD MANAGEMENT SYSTEM

Submitted by:

Name: Soubhagya Das

Roll: 27600125073

Group no. 1

Name: Vivek Singh

Roll: 27600125074

Group no. 1

Name: Bishal Adak

Roll: 27600125075

Group no. 1

Name: Rohan Bar

Roll: 27600125076

Group no. 1

The Student Record Management System is a simple Python program made to store and display basic student details and marks. In a manual method, student information is often written in registers and calculations are done by hand. This can take time and may lead to mistakes. Our program takes a student's name, roll number and marks in three subjects through the keyboard. It checks whether the marks are between 0 and 100, calculates the total and percentage, and displays a pass or fail result using a sample 40% rule. The program also allows the user to display all saved records and search for a student using the roll number. It is written in Python and uses basic concepts such as variables, data types, input/output, operators, conditions, loops, functions, lists and dictionaries. The records are stored temporarily while the program is running.

Student details and marks need to be recorded clearly so that they can be checked when required. For a small number of students, a register or notebook can be used, but finding a particular record and calculating marks repeatedly can be inconvenient. A computer program can make these tasks simpler.

The purpose of this mini-project is to practise the basic concepts taught in Programming with Python by making a small working application. The program is menu-driven, so the user can choose to add a student, display saved records, search for a student or exit the program.

Problem Statement

The project aims to reduce the effort needed to record student details and calculate marks manually. It provides a simple way to enter student information, check the marks range, calculate total and percentage, and retrieve a saved record by roll number.

Objectives

- To accept student name, roll number and marks through keyboard input.
- To store each student's information in a structured form.
- To check that each subject mark is between 0 and 100.
- To calculate total marks and percentage automatically.
- To show a pass or fail result using the example rule used in the program.
- To display all records currently stored in the program.
- To search for a student by roll number.
- To practice Python variables, data types, operators, conditions, loops, functions, lists and dictionaries.

Existing System

In a traditional system, student details and marks may be written in a paper register or entered into separate documents. Total marks and percentage may be calculated manually. This method is easy to start with, but it has some limitations:

- Calculations may take extra time and can contain human errors.
- Finding one student's details in a large register may be difficult.
- Marks may be written incorrectly if they are not checked carefully.
- Updating or comparing records can require repeated manual work.

Proposed System

The proposed system is a menu-driven Python program. When it starts, it displays four options: Add Student, Display All Students, Search Student and Exit. The user selects an option by entering its number.

- Add Student: accepts the student's details and three subject marks.
- Validation: checks whether all marks are within the range 0 to 100.
- Calculation: adds the marks and calculates the percentage.
- Result: displays Pass when the calculated percentage is at least 40; otherwise it displays Fail. This is an example rule and should be changed if the teacher specifies a different rule.
- Storage: saves the record in a list as a dictionary.
- Display/Search: reads the records currently in memory and prints the matching information.

Scope of the Project

The project covers the following functions:

- Entering a student's name and roll number.
- Entering marks for Python, Mathematics and Computer Science.
- Validating marks within the range 0–100.
- Calculating total marks out of 300 and percentage.
- Displaying the pass/fail status.
- Displaying all saved records during the current run.
- Searching for a record using a roll number.
- Providing a simple menu and an exit option.

The current version does not include permanent file/database storage, editing or deleting records, user login, a graphical interface, or a network connection.

Hardware and Software Requirements

Type	Requirement
Hardware	A basic desktop computer or laptop; keyboard and display.
Processor / RAM	Any computer capable of running a supported Python 3 version; 4 GB RAM or more is sufficient for this small program.
Operating System	Windows, Linux or macOS.
Programming Language	Python 3.
IDE / Editor	IDLE, Visual Studio Code, PyCharm or any text editor.
Database	Not required in the current version; records are stored temporarily in a Python list.
JDK	Not required because this project is written in Python, not Java.
Other Tools	Python interpreter and terminal/command prompt.

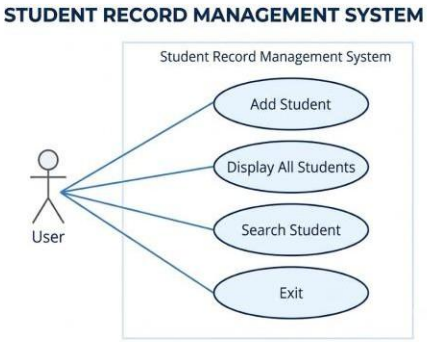
System Design

Use Case Diagram (text version)

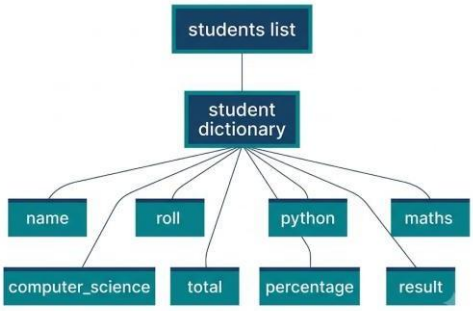
Actor: User

User action	Program response
Add Student	Accept details, validate marks, calculate result and save the record.
Display All Students	Show every record stored in the current run.
Search Student	Find and display a record matching the entered roll number.
Exit	Stop the menu loop and close the program.

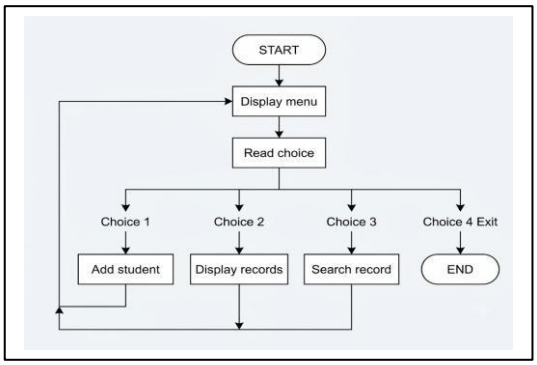
Simple use-case view:



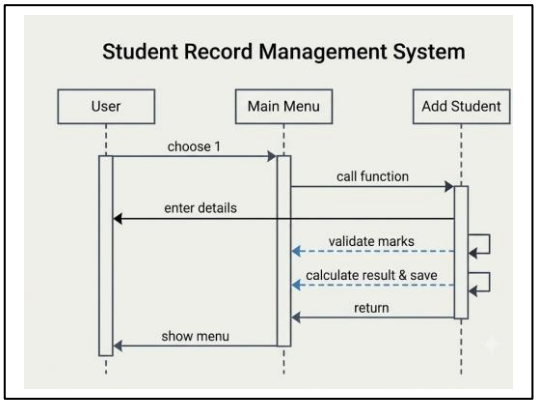
Data Structure Diagram



Activity Diagram



Sequence Diagram



Modules Description

Module / Function	Purpose
students = []	Creates an empty list to hold student records during the current program run.
add_student()	Reads details, validates marks, calculates total/percentage/result, creates a dictionary and saves it.
display_students()	Checks whether records exist and displays the details of each saved student.
search_student()	Accepts a roll number, checks each record and displays the

	matching student if found.
main()	Shows the menu repeatedly and calls the correct function based on the user's choice.
if / elif / else	Selects the operation and handles invalid menu choices.
while True / break	Repeats the menu until the user chooses Exit.

Algorithm and Flowchart

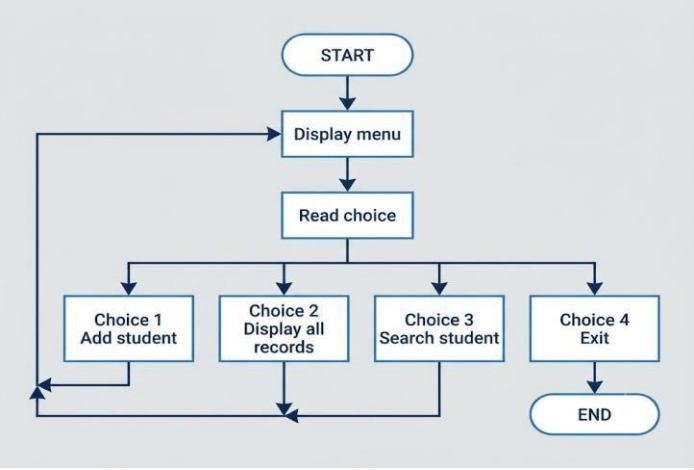
Algorithm: Add Student

1. Start the Add Student function.
2. Read the student's name and roll number.
3. Read marks for Python, Mathematics and Computer Science.
4. Check whether each mark is between 0 and 100.
5. If any mark is outside the range, display an error and return to the menu.
6. Add the three marks to calculate total marks.
7. Divide the total by 3 to calculate the percentage.
8. Set the result to Pass if percentage is at least 40; otherwise set it to Fail.
9. Store the details in a dictionary and append it to the students list.
10. Display a success message and return to the menu.

Algorithm: Search Student

11. Start the Search Student function.
12. Read the roll number to search for.
13. Check each student dictionary in the students list.
14. If the roll number matches, display the student's details and stop searching.
15. If no record matches, display 'Student record not found'.
16. Return to the main menu.

Main Menu Flowchart (text version)



Implementation

The program is written in Python. A list stores all student records, and each record is represented by a dictionary. Functions separate the main tasks, making the code easier to read and debug. The menu uses a while loop so the user can perform more than one operation in one run.

Important logic

Input and conversion: input() reads the value as text. float() converts marks into numbers so calculations can be performed.

```
marks1 = float(input("Enter Python marks (out of 100): "))
```

Validation: the condition checks that all three marks are within the allowed range.

```
if not (0 <= marks1 <= 100 and
        0 <= marks2 <= 100 and
        0 <= marks3 <= 100):
    print("Error: Marks must be between 0 and 100.")
    return
```

Calculation: the total is the sum of three subject marks. Since the maximum total is 300, dividing the total by 3 gives the percentage.

```
total = marks1 + marks2 + marks3
percentage = total / 3
```

Storage: the dictionary keeps the details together, and append() adds it to the list.

```
student = {
    "name": name,
    "roll": roll,
    "python": marks1,
    "maths": marks2,
    "computer_science": marks3,
    "total": total,
    "percentage": percentage,
    "result": result
}
students.append(student)
```

Complete Source Code

```
# Student Record Management System

students = []


def add_student():
    print("\n--- Add Student Record ---")

    name = input("Enter student name: ")
    roll = input("Enter roll number: ")

    marks1 = float(input("Enter Python marks (out of 100): "))
    marks2 = float(input("Enter Maths marks (out of 100): "))
    marks3 = float(input("Enter Computer Science marks (out of 100): "))

    # Validate marks
    if not (0 <= marks1 <= 100 and
            0 <= marks2 <= 100 and
            0 <= marks3 <= 100):
        print("Error: Marks must be between 0 and 100.")
        return

    # Calculate total and percentage
    total = marks1 + marks2 + marks3
    percentage = total / 3

    # Determine result (example pass rule: 40%)
    if percentage >= 40:
        result = "Pass"
    else:
```

```
        result = "Fail"

    # Store student details
    student = {
        "name": name,
        "roll": roll,
        "python": marks1,
        "maths": marks2,
        "computer_science": marks3,
        "total": total,
        "percentage": percentage,
        "result": result
    }

    students.append(student)
    print("\nStudent record added successfully!")


def display_students():
    print("\n--- Student Records ---")

    if len(students) == 0:
        print("No student records found.")
        return

    for student in students:
        print("\nName:", student["name"])
        print("Roll Number:", student["roll"])
        print("Python Marks:", student["python"])
        print("Maths Marks:", student["maths"])
        print("Computer Science Marks:", student["computer_science"])
        print("Total Marks:", student["total"], "/ 300")
        print("Percentage:", round(student["percentage"], 2), "%")
        print("Result:", student["result"])


def search_student():
    print("\n--- Search Student ---")

    roll = input("Enter roll number to search: ")

    for student in students:
        if student["roll"] == roll:
            print("\nStudent Found!")
            print("Name:", student["name"])
            print("Roll Number:", student["roll"])
            print("Total Marks:", student["total"], "/ 300")
            print("Percentage:", round(student["percentage"], 2), "%")
            print("Result:", student["result"])
            return

    print("Student record not found.")


def main():
    while True:
        print("\n===== STUDENT RECORD MANAGEMENT SYSTEM =====")
        print("1. Add Student")
        print("2. Display All Students")
        print("3. Search Student")
        print("4. Exit")

        choice = input("Enter your choice: ")

        if choice == "1":
            add_student()
        elif choice == "2":
            display_students()
        elif choice == "3":
            search_student()
        elif choice == "4":
            print("Thank you for using the system!")
            break
```

```
else:
    print("Invalid choice. Please enter 1, 2, 3, or 4.")
```

Results and Testing

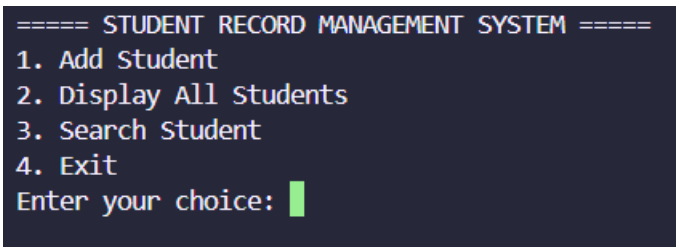
The following test cases describe the expected behaviour of the program. Run the program and check each case before final submission. Fill the Actual Result column based on your own run; do not mark a test as passed unless you have tested it.

Test Case	Input / Action	Expected Result	Actual Result
TC-01: Add valid record	Name: Rahul; roll: 101; marks: 85, 78, 92	Record saved; total 255; percentage 85%; Pass	To be filled after testing
TC-02: Invalid marks	Enter 120 for one subject	Error message; record is not saved	To be filled after testing
TC-03: Display with no records	Choose Display before adding any student	Message: No student records found	To be filled after testing
TC-04: Search existing roll	Search for a roll number already added	Student Found and summary is displayed	To be filled after testing
TC-05: Search missing roll	Search for roll number 999 when it is absent	Message: Student record not found	To be filled after testing
TC-06: Invalid menu choice	Enter 9 at the main menu	Invalid choice message appears	To be filled after testing
TC-07: Exit	Choose option 4	Thank-you message and program ends	To be filled after testing

Application Screenshots

Insert real screenshots from your own run of the program in the spaces below. Take screenshots after running the code in VS Code terminal or Command Prompt. Do not use sample screenshots from another project.

Screenshot 1: Main Menu



Screenshot 2: Add Student

```
--- Student Records ---

Name: Soubhagya Das
Roll Number: 73
Python Marks: 100.0
Maths Marks: 99.0
Computer Science Marks: 98.0
Total Marks: 297.0 / 300
Percentage: 99.0 %
Result: Pass

Name: Vivek Singh
Roll Number: 74
Python Marks: 80.0
Maths Marks: 89.0
Computer Science Marks: 79.0
Total Marks: 248.0 / 300
Percentage: 82.67 %
Result: Pass

Name: Bishal Adak
Roll Number: 75
Python Marks: 84.0
Maths Marks: 69.0
Computer Science Marks: 45.0
Total Marks: 198.0 / 300
Percentage: 66.0 %
Result: Pass

Name: Rohan Bar
Roll Number: 76
Python Marks: 85.0
Maths Marks: 45.0
Computer Science Marks: 71.0
Total Marks: 201.0 / 300
Percentage: 67.0 %
Result: Pass
```

Conclusion

In this mini-project, we created a simple Student Record Management System using Python. The program accepts student details and marks, checks the marks range, calculates total and percentage, displays a result, lists saved records and searches by roll number. While developing this project, we practised variables, data types, input/output, operators, conditions, loops, functions, lists and dictionaries. The project meets the basic learning purpose of using Python to solve a small record-management problem. Since the current version stores data only in memory, it is suitable as a beginner project and can be improved further.

Future Scope

- Save records permanently in a text file, CSV file or database.
- Add options to update and delete student records.
- Check for duplicate roll numbers before saving a record.
- Handle non-numeric marks safely using try-except.
- Add more subjects and allow the user to set the maximum marks.
- Generate a report card for each student.
- Add a graphical user interface if required.
- Add login and access control in a more advanced version.

References

- Python Software Foundation, Python 3 Documentation: https://docs.python.org/3/

- Python Software Foundation, Built-in Functions (input, float, len, round): https://docs.python.org/3/library/functions.html
- Python Software Foundation, Data Structures (lists and dictionaries): https://docs.python.org/3/tutorial/datastructures.html
- Class notes and laboratory instructions for Programming with Python (PCC-CS393), BBIT.